Equation Details

Back to List

Equation42[x ◇ y = x ◇ z]

Commentary of the equivalent Equation38[x ◇ x = x ◇ y]:

The left unar law

This law states that the magma operation is left-projection composed with squaring, namely `x◇y = S(x)` where `S: x ↦ x◇x`. Such magmas are in one-to-one correspondence with sets equipped with an endofunction (self-map).

Any law that implies this law or its dual (laws 4, 5, 13, 24, 38, 39, 41, 62, 256, 336, 374, 463, 3051, 3359, 4117 and higher-order laws) describes sets equipped with a self-map subject to a condition. The equivalence class of this law is among the largest ones (40 laws in our list of 4694 laws), and a full 57% of the 4694 laws imply either law 38 or law 39.

This law cannot hold in any non-trivial quasigroup. In a commutative magma it is equivalent to the constant law. This law together with associativity is equivalent to law 374. Magmas satisfying this law are always submagmas of linear magmas satisfying this law.

The free magma on some set Σ of generators for this law is `ℕ×Σ` with `S(m,s)=(m+1,s)`, namely `(m,s) ◇ (n,t) = (m+1,s)`.

The number of magmas of size n (modulo isomorphism) satisfying this law is OEIS sequence A001372.

(Dual equation: Equation45[x ◇ y = z ◇ y])

(Visualize implies and implied by of the equation, or see 1, 2, 3 graph edges away)

(Size of smallest non-trivial magma: 2 (Explore))

☑ Hide equivalent equations ☑ Treat conjectures as unknown ☐ Display the finite graph ☐ Show only explicit proofs

This equation implies (=>):

Implies	Does not imply	Unknown
Equation1[x = x] Try This! Show Proof	**Equation2[x = y] (+ 1495 equiv.) Try This! Show Proof**	None
Equation307[x ◇ x = x ◇ (x ◇ x)] Try This! Show Proof	**Equation3[x = x ◇ x] Try This! Show Proof**	
Equation308[x ◇ x = x ◇ (x ◇ y)] Try This! Show Proof	**Equation4[x = x ◇ y] (+ 70 equiv.) Try This! Show Proof**	
Equation309[x ◇ x = x ◇ (y ◇ x)] Try This! Show Proof	**Equation5[x = y ◇ x] (+ 70 equiv.) Try This! Show Proof**	
Equation310[x ◇ x = x ◇ (y ◇ y)] (+ 1 equiv.) Try This! Show Proof	**Equation8[x = x ◇ (x ◇ x)] Try This! Show Proof**	
Equation311[x ◇ x = x ◇ (y ◇ z)] Try This! Show Proof	**Equation9[x = x ◇ (x ◇ y)] (+ 8 equiv.) Try This! Show Proof**	

This equation is implied by (<=):

Implied by	Not implied by	Unknown by
Equation24[x = (x ◇ x) ◇ y] (+ 111 equiv.) Try This! Show Proof	**Equation1[x = x] Try This! Show Proof**	None
Equation41[x ◇ x = y ◇ z] (+ 418 equiv.) Try This! Show Proof	**Equation3[x = x ◇ x] Try This! Show Proof**	
Equation256[x = ((x ◇ x) ◇ x) ◇ y] (+ 75 equiv.) Try This! Show Proof	**Equation5[x = y ◇ x] (+ 70 equiv.) Try This! Show Proof**	
Equation374[x ◇ y = (x ◇ x) ◇ x] (+ 36 equiv.) Try This! Show Proof	**Equation8[x = x ◇ (x ◇ x)] Try This! Show Proof**	
Equation3051[x = (((x ◇ x) ◇ x) ◇ x) ◇ y] (+ 26 equiv.) Try This! Show Proof	**Equation9[x = x ◇ (x ◇ y)] (+ 8 equiv.) Try This! Show Proof**	
Equation4117[x ◇ y = ((x ◇ x) ◇ x) ◇ x] (+ 17 equiv.) Try This! Show Proof	**Equation10[x = x ◇ (y ◇ x)] (+ 5 equiv.) Try This! Show Proof**	

Equivalent Equations

Equivalent Equations:

- Equation38[x ◇ x = x ◇ y]
- Equation322[x ◇ y = x ◇ (x ◇ x)]
- Equation324[x ◇ y = x ◇ (x ◇ z)]
- Equation328[x ◇ y = x ◇ (z ◇ x)]
- Equation330[x ◇ y = x ◇ (z ◇ z)]
- Equation331[x ◇ y = x ◇ (z ◇ w)]
- Equation3305[x ◇ y = x ◇ (x ◇ (x ◇ x))]

Back to Equational Theories

Last updated at: 07/12/2025, 10:40:11 (local time), Git commit: 3980355

FIGURE 16. An example of the information displayed by the Equation Explorer for a specific equation.

14. CONCLUSIONS AND FUTURE DIRECTIONS

This project successfully demonstrated that large-scale explorations of a space of mathematical statements (in this case, the implications or non-implications between selected equational

laws) can be crowdsourced using modern collaboration platforms and proof assistants. No single tool or method was able to study the entirety of this space, and many informal proofs generated contained non-trivial errors; but there were multiple techniques that could treat significant portions of the space, and through a collaborative effort combined with the proof validation provided by *Lean*, one could synthesize these partial and fallible contributions into a complete and validated description of the entire implication graph. While this particular graph was a comparatively simple structure to analyze, we believe that this paradigm could also serve as a model for future projects devoted to exploring more sophisticated large-scale mathematical structures.

Several factors appeared to be helpful in ensuring the success of the project, including the following:

- **A clearly stated primary goal, with an end condition and precise numerical metrics to measure partial completion.** From the outset, there was a specific goal to attain, namely to completely determine and then formalize the implication graph on the original set of 4694 laws. Progress towards that goal could be measured by a number of metrics, such as the number of implications that were conjectured but unformalized, or not conjectured at all. Such metrics allowed participants to see how partial contributions, such as formalizing a certain subset of implications, advanced the project directly towards its primary goal. This is not to say that all activity was devoted solely towards this primary goal, but it did provide a coherent focus to help guide and motivate other secondary activities.
- **A highly modular project**. It was possible for any given coauthor to work on a small subset of implications and focus on a single proof technique, without needing to understand or rely upon other contributions to the project. This allowed the work to be both parallelized and decentralized; many contributors launched their own investigations broadly within the framework of the project, without needing centralized approval or coordination.
- **Low levels of required mathematical and formal prerequisites**. The problems considered in the project did not require advanced mathematical knowledge (beyond a general familiarity with abstract algebra), nor a sophisticated understanding of formal proof assistants. This permitted contributions from a broad spectrum of participants, including those without a graduate mathematical training, as well as mathematicians with no experience in proof formalization. At a technical level, it also meant that formalization of proofs into *Lean* could be done immediately once certain base definitions (such as `Magma`) were constructed. This can be compared for instance with the recent formalization of the Polynomial Freiman–Ruzsa conjecture [22], in which significant effort was expended in the first few days to settle on a suitable framework to formalize the mathematics of Shannon entropy. While some more sophisticated formal structures (such as the syntactic description of laws as pairs of words in a `FreeMagma`) were later introduced in the project, it was relatively straightforward to refactor previously written code to be compatible with these structures as they were incorporated into the project.
- **Variable levels of difficulty, and the amenability to partial progress.** Traditional mathematics projects generally involve a small number of extremely hard problems, with incomplete progress on these problems being difficult to convert into

clean partial results. In contrast, the ETP studied a large number of problems with a very broad range of difficulty, so that even if a given proof strategy did not work for a given implication, it could be the case that there was some class of easier implications for which the strategy was successful. This allowed for a means to validate such ideas, and allowed the project to build up a useful and diverse toolbox of proof techniques which became increasingly necessary to handle the final and most difficult implications in the project. It also created a dynamic in which the project initially focused on easy techniques to resolve a significant fraction of the implications, gradually transitioning into more sophisticated methods that focused on a much smaller number of outstanding implications that had proven resistant (or even "immune") to all easier approaches.

- **Centralized and standardized platforms for discussion, project management, and validation.** While the project was decentralized at the level of the participant, there was a centralized location (a channel[35] on the Lean Zulip) to discuss all aspects of the project, as well as a centralized repository[36] to track all contributions and outstanding issues, a centralized blueprint[37] to describe technical details of proofs to be formalized, and a single formal language (*Lean*) to validate all contributions. A significant portion of the activity in the early stages of the project was devoted to setting out the standards and workflows for handling both the discussion and the contributions, in particular setting up a contributions page[38] and adopting a code of conduct[39]. This gave some structure and predictability to what might otherwise be a chaotic effort.
- **Development of custom visualization tools.** As discussed in Section 12, several tools were developed (in part with AI assistance) to help visualize and navigate the implication graph while it was in a partial stage of development, allowing for participants to independently identify problems to work on, and to validate and use the contributions of other participants even before they were fully formalized. For instance, a participant could propose a finite counterexample to an implication by posting a link to the magma in *Finite Magma Explorer*, allowing for immediate validation of the counterexample, or use *Equation Explorer* or *Graphiti* to observe some interesting phenomenon in the implication graph that other participants could reproduce and study.
- **Applicability of existing software tools.** As described in Section 7, many of the implications in the ETP were amenable to application of "off-the-shelf" automated theorem provers (ATPs); while some trial and error was needed to determine good choices of parameters, these tools could largely be applied directly to the project without extensive customization. (However, the later transcription of ATP output into Lean was sometimes non-trivial.)
- **Receptiveness to new techniques and tools.** Crucially, the methods used to make progress on the project were not specified in advance, and contributions from participants with new ideas, techniques, or software tools that were not initially

[35] `https://leanprover.zulipchat.com/#narrow/channel/458659-Equational`
[36] `https://github.com/teorth/equational_theories`
[37] `https://teorth.github.io/equational_theories/blueprint/`
[38] `https://github.com/teorth/equational_theories/blob/main/CONTRIBUTING.md`
[39] `https://github.com/teorth/equational_theories/blob/main/CODE_OF_CONDUCT.md`